Day 2 - Length, Middle & Reversal of LinkedList

ITSRUNTYM

Length

Goal: count how many nodes are in the list.
Approaches

1. **Iterative (recommended)** — one pass, constant extra space.

Iterative (simple)

Idea: start at head, follow next while incrementing a counter.
Complexity: time O(n), space O(1).
Example list: 10 → 20 → 30 → null
Trace: count 0 → visit 10 (count=1) → visit 20 (2) → visit 30 (3) → null → return 3.

Middle

Goal: find the middle node. Clarify: for even-length lists there are two "middles" — we can choose **first (left)** or **second (right)** middle. Many problems (e.g., LeetCode 876) return the second middle.

Approaches

1. **Two-pass**: count length n, then walk n/2 steps (gives the *second* middle if you use n/2). Time O(n), space O(1).
2. **Fast & slow pointers (single pass)**:
    - **Standard: slow=head, fast=head. While fast!=null && fast.next!=null do slow=slow.next; fast=fast.next.next;. This returns the second middle for even n.**
    - **Variant for first middle: start fast = head.next (instead of head); same loop stops with slow at the first middle.**

Complexity: time O(n), space O(1).
Examples

- Odd length 5: nodes indices 0..4 → middle index 2. Fast-slow returns node at index 2.
- Even length 4:
    - fast=head default → slow ends at index 2 (second middle).

- fast=head.next → slow ends at index 1 (first middle).

Reversal

Goal: reverse the list so head points to the previous tail.

Approaches

1. **Iterative in-place (recommended)** — use three pointers: prev, curr, next.

Iterative (in-place)

Idea: walk through list, flip curr.next to prev at each step.
Pseudocode loop:

```
prev = null
curr = head
while curr != null:
    next = curr.next
    curr.next = prev
    prev = curr
    curr = next
head = prev
```

Complexity: time O(n), space O(1).

Example trace for 1 → 2 → 3 → null:

- Start: prev=null, curr=1
- Step1: next=2; 1.next = null; prev=1; curr=2 => list fragment: 1→null, remaining 2→3
- Step2: next=3; 2.next = 1; prev=2; curr=3 => 2→1→null, remaining 3
- Step3: next=null; 3.next = 2; prev=3; curr=null => 3→2→1→null
- done; new head = prev (3)

#	Problem Name & Link	Core Concept	Difficulty	Key Techniques
1	1290. Convert Binary Number in a Linked List to Integer	Length (traversal)	Easy	Traverse list, accumulate value
2	876. Middle of the Linked List	Middle	Easy	Fast & slow pointers
3	234. Palindrome Linked List	Middle + Reverse	Easy	Find middle, reverse second half, compare
4	2095. Delete the Middle Node of a Linked List	Middle	Medium	Fast & slow pointers, delete node
5	725. Split Linked List in Parts	Length	Medium	Find length, split into equal parts
6	206. Reverse Linked List	Reverse	Easy	Iterative & recursive reversal
7	92. Reverse Linked List II	Reverse (sublist)	Medium	Reverse between positions m & n
8	25. Reverse Nodes in k-Group	Reverse (group)	Hard	Reverse in groups of size k
9	19. Remove Nth Node From End of List	Length / Middle	Medium	Two-pass (length) or one-pass (fast-slow)
10	61. Rotate List	Length + Pointer Adjustments	Medium	Find length, reconnect tail to head